

Guía Técnica de Despliegue — Portal USEBEQ

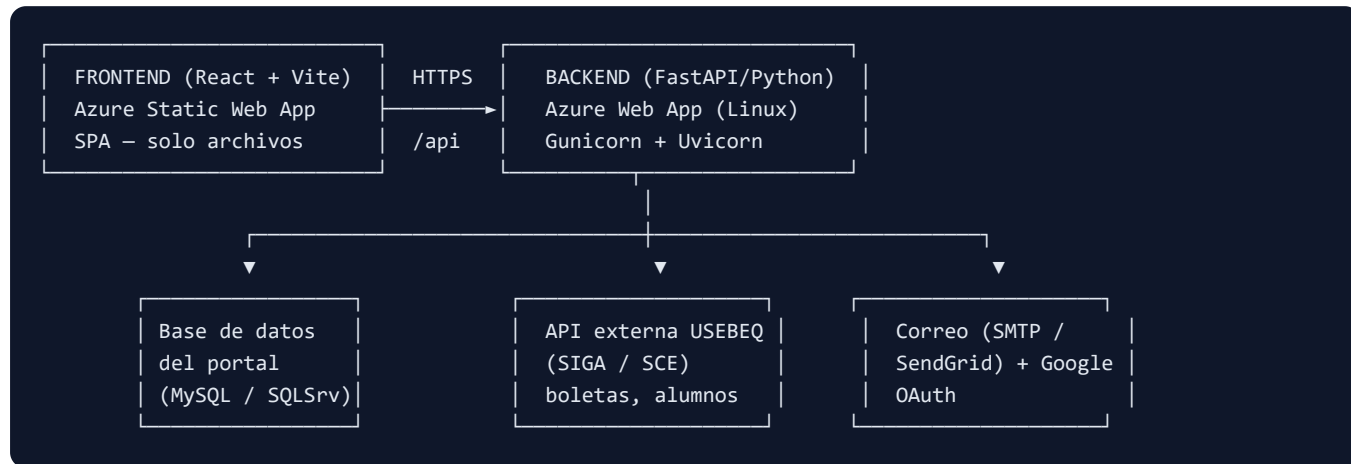
Dirigido a: Área de Tecnologías de la USEBEQ (personal técnico / DevOps). **Objetivo:** Instalar y poner en operación el **Portal para Padres de Familia** (frontend + backend + base de datos) en la infraestructura de la USEBEQ. **Versión del documento:** 1.0 — Junio 2026

Índice

1. Arquitectura general
2. Repositorios de código
3. Prerrequisitos
4. ⚠ Decisión de base de datos (MySQL vs SQL Server)
5. Despliegue del Backend (FastAPI)
6. Base de datos del portal
7. Despliegue del Frontend (React + Vite)
8. Integraciones externas
9. Verificación post-despliegue
10. Recomendaciones de seguridad (antes de entregar)
11. Mantenimiento y solución de problemas

1. Arquitectura general

El Portal está compuesto por dos aplicaciones independientes y varios servicios externos:



Componente	Tecnología	Servicio Azure sugerido
Frontend	React 19 + Vite + Tailwind CSS	Azure Static Web App
Backend	Python 3.11 + FastAPI + SQLAlchemy	Azure Web App (Linux, Python 3.11)
Base de datos del portal	MySQL (<i>actual</i>) / SQL Server (<i>ver sección 4</i>)	Azure Database for MySQL o SQL Server institucional
API académica	API REST externa de USEBEQ (SIGA/SCE)	Servicio existente de USEBEQ
Correo	SMTP / SendGrid	Servicio de correo institucional
Autenticación social	Google OAuth 2.0	Google Cloud Console

El backend consulta los datos académicos (alumnos, boletas, catálogos) a través de la **API externa de USEBEQ**; la base de datos del portal solo almacena lo propio de la plataforma (usuarios, vínculos padre-alumno, trámites, avisos, tokens).

2. Repositorios de código

Repositorio	URL	Rama productiva
Frontend	https://github.com/Ajauregui69/portal-usebeq-frontend	main
Backend	https://github.com/Ajauregui69/portal-usebeq-backend	main

Ambos repositorios están **actualizados y sincronizados** con su rama `main`.

Entrega del código a la USEBEQ (elegir una opción):

1. **Invitar como colaborador:** en cada repo → *Settings* → *Collaborators* → *Add people* (acceso de lectura para Tecnologías).
2. **Transferir a la organización de la USEBEQ:** *Settings* → *General* → *Transfer ownership*.
3. **Clonado / copia directa:**

```
git clone https://github.com/Ajauregui69/portal-usebeq-frontend.git
git clone https://github.com/Ajauregui69/portal-usebeq-backend.git
```

🔒 Antes de entregar el repositorio, revisar la [sección 10 \(Seguridad\)](#): hay credenciales que deben rotarse.

3. Prerrequisitos

- Cuenta / suscripción de **Azure** activa de la USEBEQ.
- Cuenta de **GitHub** con acceso a ambos repositorios (para el CI/CD).
- **Node.js 18+** (para build local del frontend, si se requiere).
- **Python 3.11+** (para pruebas locales del backend).
- Acceso a la **base de datos** destino (MySQL o SQL Server) y a la **API externa de USEBEQ**.
- Credenciales de **correo** (SMTP institucional o SendGrid) y de **Google OAuth**.

4. ⚠️ Decisión de base de datos (MySQL vs SQL Server)

Este es el punto más importante a definir antes del despliegue.

El backend usa **SQLAlchemy**, por lo que soporta tanto MySQL como SQL Server cambiando la cadena `DATABASE_URL`. Sin embargo, el estado **actual** del proyecto está hecho para **MySQL**:

- El archivo `.env` usa `DATABASE_URL=mysql+pymysql://...`
- `requirements.txt` incluye el driver `pymysql` (MySQL), **no** incluye driver de SQL Server.
- El script `create_tables_produccion.sql` está escrito en **sintaxis MySQL** (`AUTO_INCREMENT`, `IF NOT EXISTS`, `ENGINE`, etc.), que **no se ejecuta tal cual en SQL Server**.

Opción A — Desplegar con MySQL (camino más directo)

Sin cambios de código. Solo crear una base de datos MySQL (Azure Database for MySQL u otra) y apuntar `DATABASE_URL`.

```
DATABASE_URL=mysql+pymysql://usuario:password@host/portal_usebeq?charset=utf8mb4
```

Opción B — Desplegar con SQL Server (BD institucional)

Requiere ajustes:

1. Agregar el driver a `requirements.txt`:

```
pyodbc==5.1.0
```

2. Instalar el **ODBC Driver 17/18 for SQL Server** en el servidor.
3. Cambiar la cadena de conexión:

```
DATABASE_URL=mssql+pyodbc://usuario:password@host/USEBEQ_DB?driver=ODBC+Driver+17+for+SQL+Server
```

4. **Convertir** `create_tables_produccion.sql` a sintaxis T-SQL (cambiar `AUTO_INCREMENT` → `IDENTITY`, `IF NOT EXISTS` → comprobación con `OBJECT_ID`, tipos `DATETIME`, etc.).

✅ **Recomendación:** definir con Tecnologías qué motor se usará. Si la USEBEQ requiere obligatoriamente SQL Server, considerar este trabajo de adaptación dentro del cronograma de liberación.

5. Despliegue del Backend (FastAPI)

5.1 Crear la Web App en Azure

1. Azure Portal → **Create a resource** → **Web App**.

Campo	Valor sugerido
Resource Group	rg-portal-usebeq
Name	portal-usebeq-backend (único)
Publish	Code
Runtime stack	Python 3.11
Operating System	Linux
Region	Mexico Central
Pricing plan	Basic B1 (pruebas) / Standard S1 (producción)

5.2 Variables de entorno (Configuration → Application settings)

Variable	Descripción
DATABASE_URL	Cadena de conexión a la BD (ver sección 4)
SECRET_KEY	Clave secreta JWT (cadena larga y aleatoria)
ALGORITHM	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	30
BACKEND_CORS_ORIGINS	["https://<frontend>.azurestaticapps.net"]
FRONTEND_URL	URL pública del frontend
MAIL_USERNAME / MAIL_PASSWORD	Credenciales SMTP
MAIL_FROM	noreply@usebeq.edu.mx
MAIL_SERVER / MAIL_PORT	Servidor SMTP y puerto (587)
MAIL_FROM_NAME	Portal USEBEQ
SENDGRID_API_KEY	API key de SendGrid (si se usa)
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	Credenciales Google OAuth
GOOGLE_REDIRECT_URI	https://<backend>.azurewebsites.net/api/v1/auth/google/callback
USEBEQ_API_BASE_URL	URL de la API externa de USEBEQ (alumnos/boletas)
USEBEQ_AUTH_API_URL	URL de autenticación de la API de USEBEQ
USEBEQ_API_EMAIL / USEBEQ_API_PASSWORD	Credenciales de servicio de la API de USEBEQ

USEBEQ_API_*, GOOGLE_*, SENDGRID_* y FRONTEND_URL tienen valores por defecto en `app/core/config.py`. **Deben sobrescribirse** con los valores de producción de la USEBEQ.

5.3 Comando de inicio (Configuration → General settings → Startup Command)

```
gunicorn -w 2 -k uvicorn.workers.UvicornWorker app.main:app --bind 0.0.0.0:8000 --timeout 120
```

El archivo `startup.txt` del repositorio ya contiene este comando (con 1 worker). Ajustar `-w` según el plan/CPU contratado.

5.4 Despliegue automático (CI/CD)

El repositorio ya incluye el workflow `.github/workflows/main_portal-usebeq-backend-app-150126.yml`.

1. Descargar el **Publish Profile** de la Web App (Overview → *Download publish profile*).
2. En GitHub → *Settings* → *Secrets and variables* → *Actions* → crear el secreto `AZURE_WEBAPP_PUBLISH_PROFILE` con su contenido.
3. Ajustar el nombre de la app (`app-name`) en el workflow si cambió.
4. Cada `push` a `main` desplegará automáticamente.

5.5 Prueba local (opcional)

```
python -m venv venv
source venv/bin/activate          # Linux/Mac
venv\Scripts\activate            # Windows
pip install -r requirements.txt
cp .env.example .env             # editar con credenciales
uvicorn app.main:app --reload --port 8000
```

Documentación interactiva de la API: `http://localhost:8000/api/v1/docs` (Swagger).

6. Base de datos del portal

1. Crear la base de datos (ejemplo MySQL):

```
CREATE DATABASE portal_usebeq CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

2. **Hacer respaldo** antes de cualquier ejecución.
3. Ejecutar el script de creación de tablas incluido en el backend:

```
mysql -u <usuario> -p portal_usebeq < create_tables_produccion.sql
```

(Si se usa SQL Server, primero convertir el script — ver [sección 4.](#))

4. Verificar la conectividad del backend a la BD (firewall, IP permitida, cadena de conexión).

Las tablas con prefijo `SCE*` / `PP_*` mencionadas en documentación antigua corresponden a la **BD institucional/API externa de USEBEQ**, no a la BD propia del portal. La BD del portal contiene tablas como `pp_usuarios`, `pp_alumnos`, `pp_token`, `tramite_*`, etc.

7. Despliegue del Frontend (React + Vite)

7.1 Crear la Static Web App en Azure

1. Azure Portal → **Create a resource** → **Static Web App**.

Campo	Valor
Resource Group	rg-portal-usebeq
Name	portal-usebeq-frontend
Plan type	Free (pruebas) / Standard (producción)
Region	Mexico Central
Source	GitHub → repo portal-usebeq-frontend , rama main
Build Presets	Vite
App location	/
Api location	(vacío)
Output location	dist

7.2 Variables de entorno (en el workflow de GitHub Actions)

En Vite las variables se inyectan **en tiempo de build**, dentro del workflow `.github/workflows/azure-static-web-apps-*.yaml`, **no** en Azure Portal. Deben empezar con `VITE_`.

```
env:
  VITE_API_URL: https://<backend>.azurewebsites.net/api/v1
  VITE_GOOGLE_AUTH_ENABLED: "true"
```

7.3 Enrutamiento SPA

El archivo `staticwebapp.config.json` (ya incluido) redirige todas las rutas a `index.html` para que funcione el enrutado de React Router:

```
{ "navigationFallback": { "rewrite": "/index.html" } }
```

7.4 Dominio personalizado (opcional)

Static Web App → **Custom domains** → **+ Add** → registrar el dominio (ej. `portal.usebeq.edu.mx`) y configurar el registro **CNAME/TXT** en el DNS de la USEBEQ.

7.5 Build local (opcional)

```
npm install
npm run build      # genera la carpeta dist/
npm run preview    # previsualizar el build
```

8. Integraciones externas

8.1 API externa de USEBEQ (SIGA / SCE)

El backend obtiene un **token** de la API de autenticación de USEBEQ y lo refresca automáticamente (scheduler en `app/services/token_scheduler.py`). Configurar `USEBEQ_AUTH_API_URL`, `USEBEQ_API_BASE_URL`, `USEBEQ_API_EMAIL`, `USEBEQ_API_PASSWORD`.

8.2 Google OAuth

En Google Cloud Console → credenciales OAuth 2.0:

- **Authorized redirect URI:** `https://<backend>.azurewebsites.net/api/v1/auth/google/callback`
- Para desactivar el botón de Google, poner `VITE_GOOGLE_AUTH_ENABLED="false"` en el frontend.

8.3 Correo

Configurar SMTP institucional (`MAIL_*`) y/o `SENDGRID_API_KEY` para correos transaccionales (activación de cuenta, recuperación de contraseña).

9. Verificación post-despliegue

Lista de verificación una vez desplegado todo:

- ☐ El backend responde en `https://<backend>.azurewebsites.net/api/v1/docs` (Swagger).
- ☐ El frontend carga la página de inicio sin errores.
- ☐ Se puede **registrar** un usuario y llega el **correo de activación**.
- ☐ Se puede **iniciar sesión** y se muestra el panel principal.
- ☐ Se puede **vincular un alumno** (valida la conexión con la API de USEBEQ).
- ☐ Se puede **descargar una boleta** en PDF.
- ☐ No hay errores de **CORS** (revisar `BACKEND_CORS_ORIGINS`).

Aspecto esperado del portal funcionando:



PORTAL PARA PADRES DE FAMILIA

Bienvenido de nuevo

Correo Electrónico

Contraseña

[Iniciar Sesión](#)

[O continúa con](#)

[Iniciar sesión con Google](#)

[¿Olvidaste tu contraseña?](#)

[¿No tienes cuenta? Regístrate aquí](#)

Portal de Padres de Familia USEBEQ — Querétaro, México



Servicios ▾

JG Juan Carlos Gómez

Perfil

Salir

Bienvenido, Juan Carlos

Portal de Padres de Familia USEBEQ

+ Vincular Estudiante



Estudiantes
2



Activos
2



Vinculados
0

Mis Estudiantes

RENÉ GÓMEZ PÉREZ
CURP: GQPR160106HQLRNA6
INSCRITO

SOFÍA GÓMEZ PÉREZ
CURP: GQPS180322MQTMRFA1
INSCRITO

INFORMACIÓN ACADÉMICA

Nivel	Grado
PRIMARIA	4°
Grupo	Ciclo
A	2025-2026

[Ver Detalle](#)


[Descargar Boleta](#)

INFORMACIÓN ACADÉMICA

Nivel	Grado
PREESCOLAR	3°
Grupo	Ciclo
B	2025-2026

[Ver Detalle](#)


[Descargar Boleta](#)

Servicios del Portal



Consulta e Impresión
Consulta calificaciones por CURP



Preguntas Frecuentes
Respuestas a dudas comunes



Avisos
Información relevante para padres



Documentos Normativos
Acuerdos y normas de control escolar



Verificación de Documentos
Verifica autenticidad de boletas



Becas
Información sobre becas disponibles



Buzón de Padres
Envía documentos y consultas

10. Recomendaciones de seguridad (antes de entregar)

⚠ El repositorio del backend contiene **credenciales reales** que deben tratarse antes de la liberación:

1. **Rotar** y mover a variables de entorno (no dejarlas en código/ `.env`):
 - `GOOGLE_CLIENT_SECRET`
 - `SENDGRID_API_KEY`
 - `USEBEQ_API_EMAIL` / `USEBEQ_API_PASSWORD` (actualmente con valores por defecto en `app/core/config.py`).
2. Generar un `SECRET_KEY` nuevo y fuerte para producción.
3. Confirmar que `.env` está en `.gitignore` (ya lo está) y **no** versionar secretos.
4. **No** usar `OAUTHLIB_INSECURE_TRANSPORT` en producción (solo desarrollo local con HTTP).
5. Restringir `BACKEND_CORS_ORIGINS` exclusivamente al dominio del frontend de producción.
6. Servir todo bajo **HTTPS** (Azure lo provee por defecto).

11. Mantenimiento y solución de problemas

Síntoma	Revisar
Error 500 en el backend	Azure → Web App → <i>Log stream / Diagnose and solve problems</i> .
La BD no conecta	Firewall/IP permitida, <code>DATABASE_URL</code> , driver instalado (pymysql/pyodbc).
Error de CORS	<code>BACKEND_CORS_ORIGINS</code> debe incluir la URL exacta del frontend (formato JSON array).
Variables no se cargan	Reiniciar la Web App tras cambiarlas; verificar que no haya espacios extra.
Build del frontend falla	GitHub → Actions → revisar logs; variables deben iniciar con <code>VITE_</code> .
Error 404 en rutas del frontend	Verificar <code>staticwebapp.config.json</code> (<code>navigationFallback</code>).
No llegan correos	Revisar <code>MAIL_*</code> / <code>SENDGRID_API_KEY</code> y reputación del remitente.
Falla la consulta de alumnos/boletas	Verificar <code>USEBEQ_API_*</code> y que el token se esté refrescando (logs del scheduler).

Documentos relacionados en el repositorio

- `MANUAL_USUARIO.md` — Manual de usuario final (este paquete).
- `MANUAL_AZURE_STATIC_WEB_APP.md` — Detalle de despliegue del frontend en Azure.
- `MANUAL_AZURE_WEB_APP_PYTHON.md` (backend) — Detalle de despliegue del backend en Azure.
- `MANUAL_BASE_DE_DATOS.md` (backend) — Detalle de la base de datos.
- `MANUAL_GOOGLE_OAUTH_GMAIL_API.md` / `GOOGLE_INTEGRATION_GUIDE.md` (backend) — Integración con Google.
- `USEBEQ_API_ENDPOINTS.md` (backend) — Referencia de la API externa de USEBEQ.

Guía técnica de despliegue — Portal USEBEQ. Área de Tecnologías.